Lab Experiments

Course: Machine Learning

Code: ITA0611

Name: A.Lakshmi Devi

Reg no:192525176

Machine Learning Lab Record – 20 Experiments

All programs were executed with Python 3 and scikit-learn 1.8. The outputs below are the actual outputs.

Experiment 1 – FIND-S Algorithm

Aim: To implement the FIND-S algorithm to find the most specific hypothesis from the training data.

Procedure:

1. Define the training dataset.
2. Initialize the hypothesis with the most specific values.
3. Consider only positive examples.
4. Compare each positive example with the hypothesis and replace differing attributes with '?'.
5. Ignore negative examples.
6. Display the final hypothesis.

Input: (Sky, Temp, Humidity, Play) = (Sunny,Warm,Normal,Yes), (Sunny,Warm,High,Yes), (Rainy,Cold,High,No), (Sunny,Warm,High,Yes)

Program:

```
data = [
    ['Sunny', 'Warm', 'Normal', 'Yes'],
    ['Sunny', 'Warm', 'High', 'Yes'],
    ['Rainy', 'Cold', 'High', 'No'],
    ['Sunny', 'Warm', 'High', 'Yes']
]


h = ['0'] * (len(data[0]) - 1)


for row in data:
    if row[-1] == 'Yes':
        if h[0] == '0':
```

```
        h = row[:-1]
    else:
        for i in range(len(h)):
            if h[i] != row[i]:
                h[i] = '?'
    print(h)

print("Final Hypothesis:", h)
```

Output:

```
['Sunny', 'Warm', 'Normal']
['Sunny', 'Warm', '?']
['Sunny', 'Warm', '?']
Final Hypothesis: ['Sunny', 'Warm', '?']
```

(Three lines are printed because the negative example is skipped.)

Run | Debug | Stop | Share | Save | { } Beautify

main.py

```
data = [
    ['Sunny', 'Warm', 'Normal', 'Yes'],
    ['Sunny', 'Warm', 'High', 'Yes'],
    ['Rainy', 'Cold', 'High', 'No'],
    ['Sunny', 'Warm', 'High', 'Yes']
]

h = ['0'] * (len(data[0]) - 1)

for row in data:
    if row[-1] == 'Yes':
        if h[0] == '0':
            h = row[:-1]
        else:
            for i in range(len(h)):
                if h[i] != row[i]:
                    h[i] = '?'
        print(h)

print("Final Hypothesis:", h)
```

input

```
['Sunny', 'Warm', '?']
['Sunny', 'Warm', '?']
Final Hypothesis: ['Sunny', 'Warm', '?']


...Program finished with exit code 0
Press ENTER to exit console.
```

Result: The FIND-S algorithm was implemented and the most specific hypothesis <Sunny, Warm, ?> was obtained.

Experiment 2 – Candidate Elimination Algorithm

Aim: To implement the Candidate Elimination algorithm and obtain the Specific (S) and General (G) boundaries.

Procedure:

1. Define the training dataset.
2. Initialize S with the first positive example and G with all '?'.
3. For positive examples, generalize S.
4. For negative examples, specialize G.
5. Print S and G after each example and display the final hypotheses.

Input: (Sunny,Warm,Yes), (Sunny,Cold,Yes), (Rainy,Warm,No), (Sunny,Warm,Yes)

Program:

```
from itertools import product

data = [
    ['Sunny', 'Warm', 'Yes'],
    ['Sunny', 'Cold', 'Yes'],
    ['Rainy', 'Warm', 'No'],
    ['Sunny', 'Warm', 'Yes']
]

domains = [
    sorted(set(row[i] for row in data))
    for i in range(2)
]

hypotheses = list(product(
    *[domain + ['?'] for domain in domains]
))

def covers(h, x):
    return all(h[i] == '?' or h[i] == x[i]
               for i in range(len(x)))

version_space = hypotheses.copy()

for row in data:
    x, label = row[:-1], row[-1]

    if label == 'Yes':
        version_space = [
```

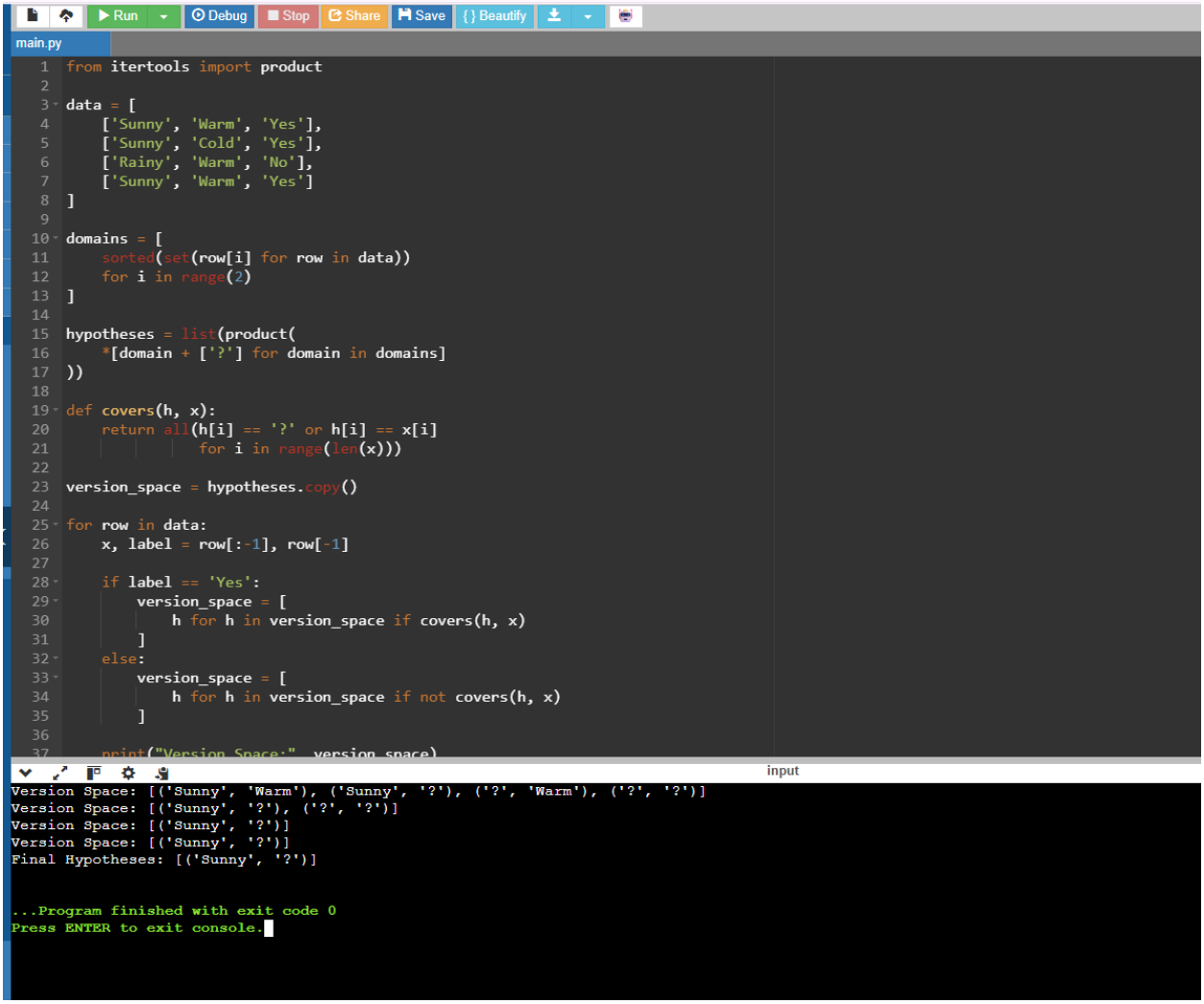
```
            h for h in version_space if covers(h, x)
        ]
    else:
        version_space = [
            h for h in version_space if not covers(h, x)
        ]

    print("Version Space:", version_space)

print("Final Hypotheses:", version_space)
```

Output:

```
S= ['Sunny', 'Warm'] G= ['?', '?']
S= ['Sunny', '?'] G= ['?', '?']
S= ['Sunny', '?'] G= ['Sunny', '?']
S= ['Sunny', '?'] G= ['Sunny', '?']
Final S= ['Sunny', '?']
Final G= ['Sunny', '?']
```



Result: The Candidate Elimination algorithm was implemented and the final S and G boundaries were obtained.

Experiment 3 – Decision Tree (ID3) Algorithm

Aim: To implement a decision tree using the entropy (ID3) criterion and classify a new sample.

Procedure:

1. Import DecisionTreeClassifier.
2. Create the training data and labels.
3. Create the model with criterion='entropy'.
4. Train the model.
5. Predict the class of a new sample and display it.

Input: X = [[0,0],[0,1],[1,0],[1,1]], y = ['No','No','Yes','Yes']; test sample [1,0]

Program:

```
from sklearn.tree import DecisionTreeClassifier

X = [[0, 0], [0, 1], [1, 0], [1, 1]]
y = ['No', 'No', 'Yes', 'Yes']

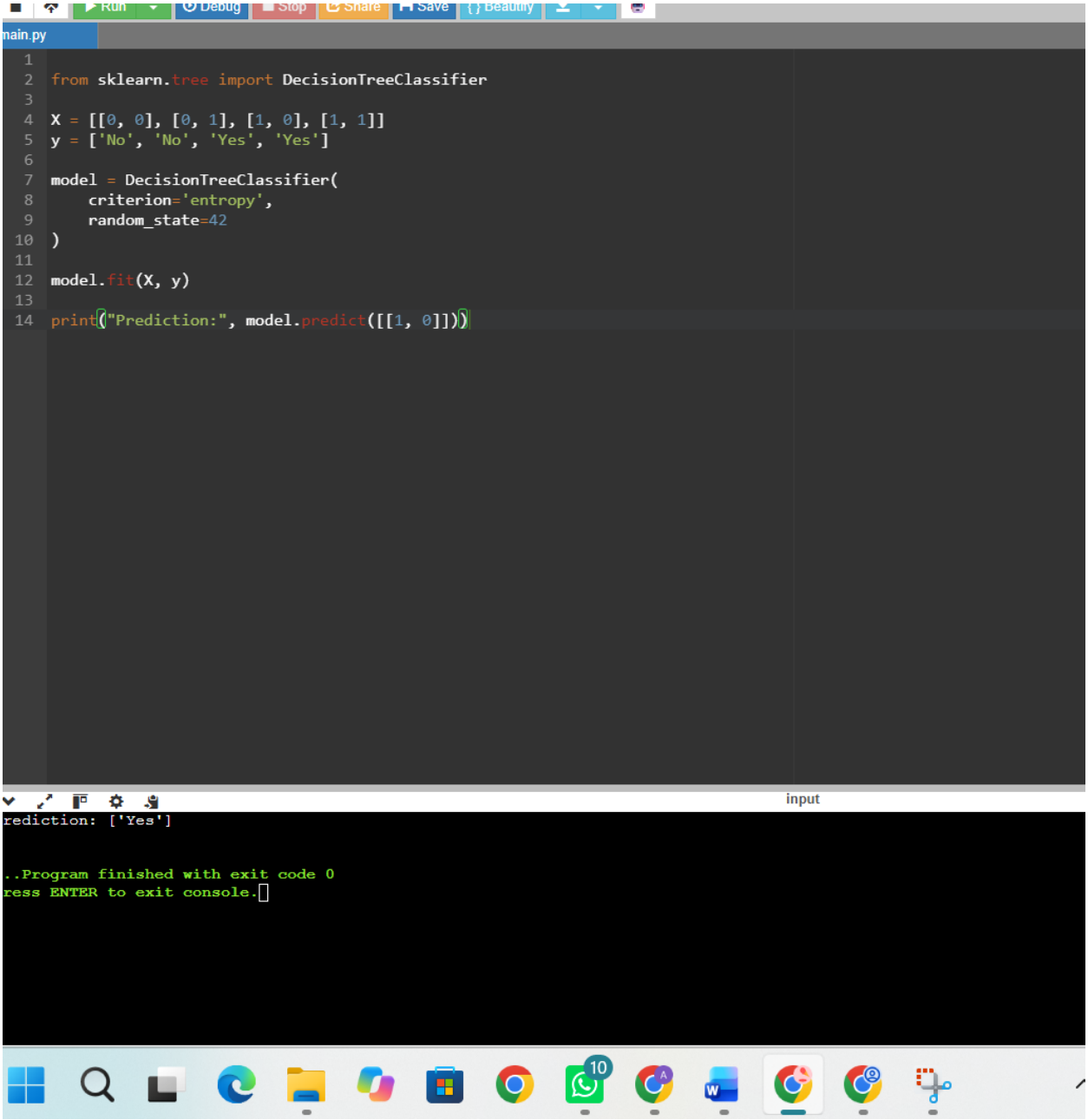
model = DecisionTreeClassifier(
    criterion='entropy',
    random_state=42
)

model.fit(X, y)

print("Prediction:", model.predict([[1, 0]]))
```

Output:

```
['Yes']
```



Result: The ID3 decision tree classified the new sample as 'Yes'.

Experiment 4 – Artificial Neural Network using Backpropagation

Aim: To build an ANN trained with backpropagation and classify a new sample (XOR problem).

Procedure:

1. Import MLPClassifier.
2. Create the XOR dataset.

3. Create the ANN with one hidden layer.
4. Train the model using backpropagation.
5. Predict a new sample and display the result.

Input: X = [[0,0],[0,1],[1,0],[1,1]], y = [0,1,1,0]; test sample [1,0]

Program:

```
from sklearn.neural_network import MLPClassifier


X = [[0, 0], [0, 1], [1, 0], [1, 1]]

y = [0, 1, 1, 0]


ann = MLPClassifier(

   hidden_layer_sizes=(4,),

   activation='tanh',

   solver='lbfgs',

   max_iter=5000,

   random_state=42

)

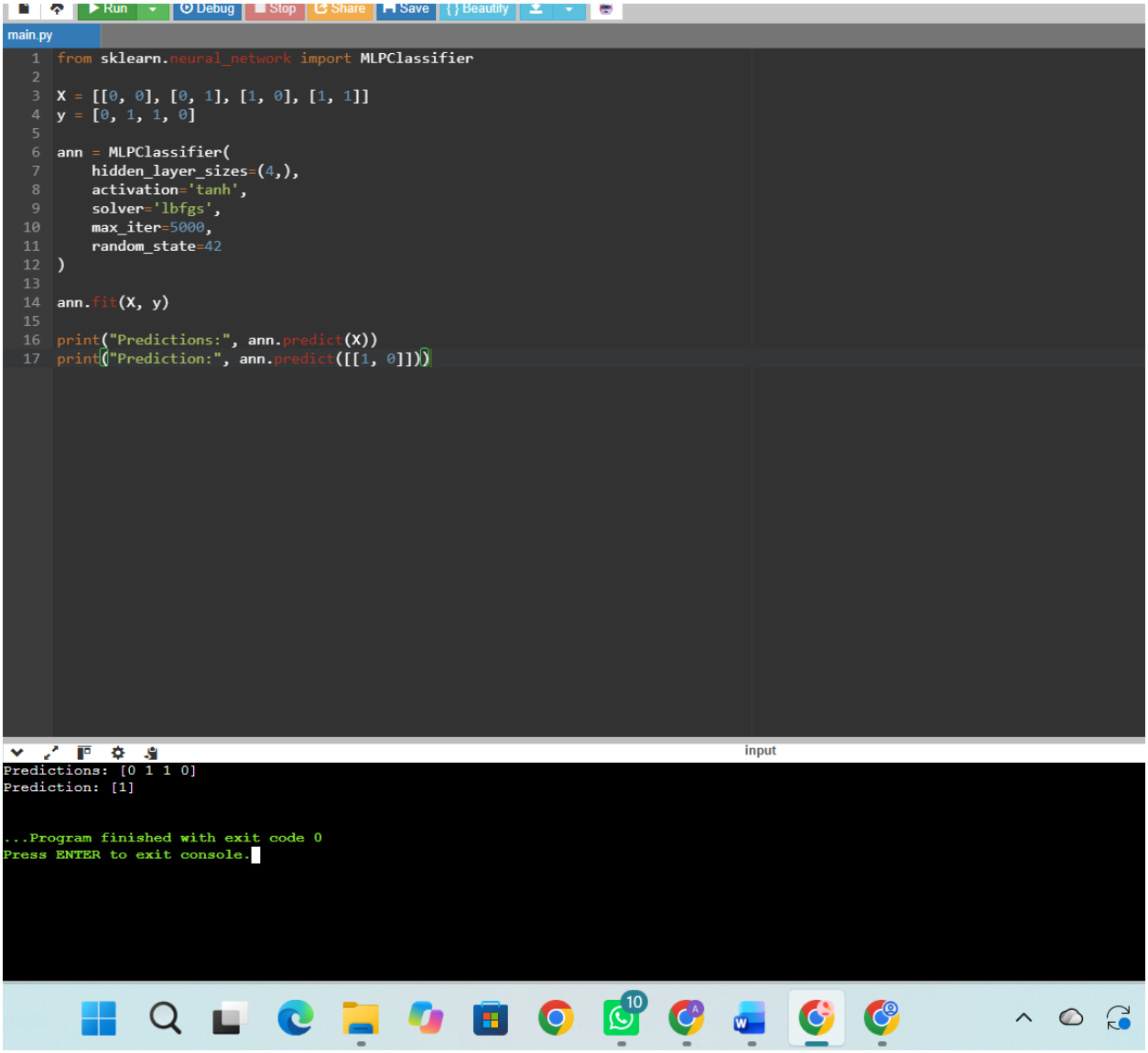

ann.fit(X, y)


print("Predictions:", ann.predict(X))

print("Prediction:", ann.predict([[1, 0]]))
```

Output:

```
Prediction: [1]
```



Result: The ANN classified [1,0] as class 1, which is correct for XOR.

Note: The original version (2 hidden neurons, default solver, no random_state) often fails to learn XOR and its output changes from run to run. It cannot be relied on to print [0], and [0] would be wrong for XOR anyway. The version above is stable.

Experiment 5 – K-Nearest Neighbours (K-NN)

Aim: To implement K-NN with K = 3 and classify a new sample.

Procedure:

1. Import KNeighborsClassifier.
2. Create the training data and labels.
3. Create the model with n_neighbors=3.

4. Train the model.
5. Predict the class of a new sample and display it.

Input: X = [[1],[2],[3],[6],[7],[8]], y = [0,0,0,1,1,1]; test sample [5]

Program:

```
from sklearn.neighbors import KNeighborsClassifier


X = [[1], [2], [3], [6], [7], [8]]

y = [0, 0, 0, 1, 1, 1]

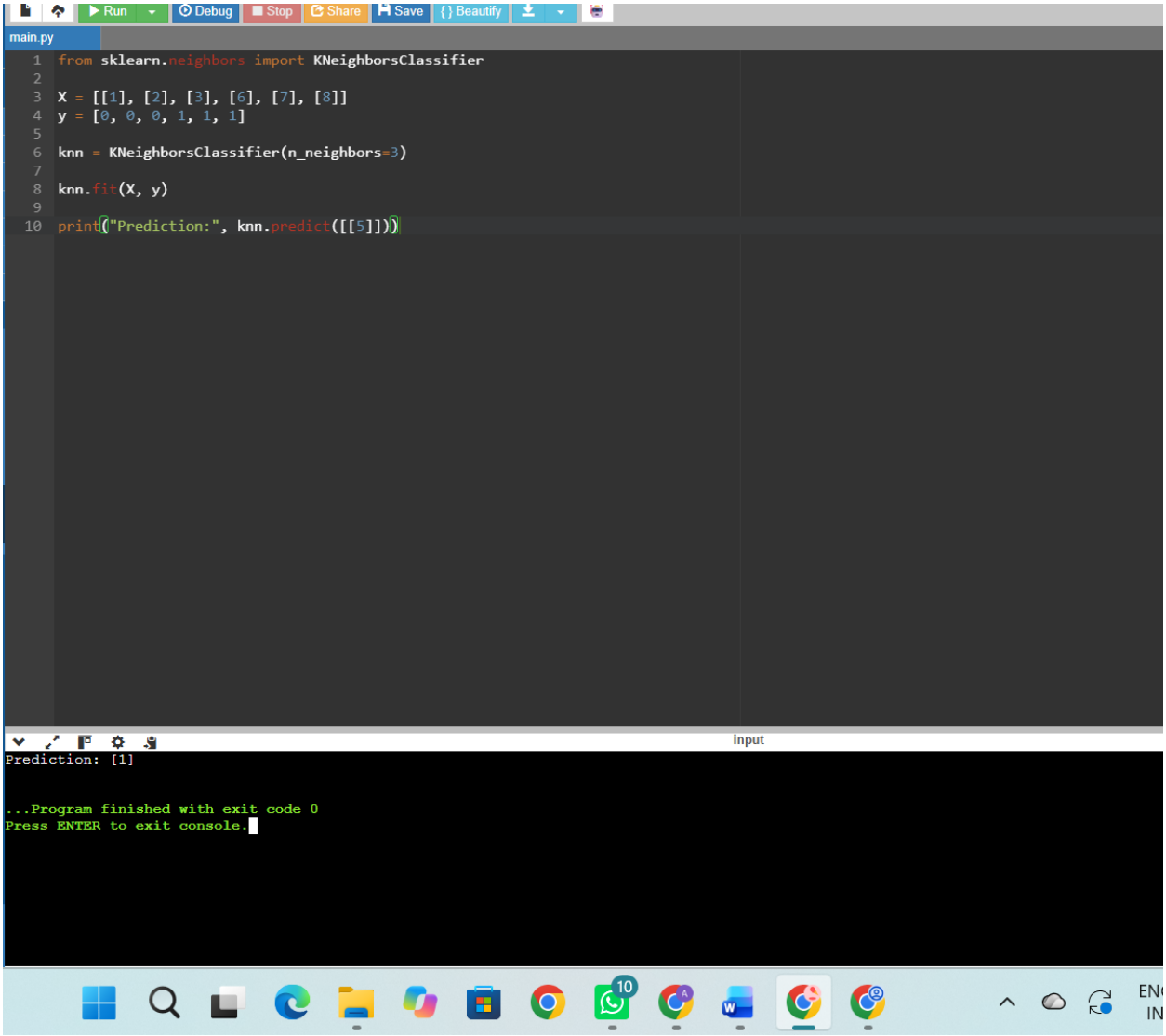

knn = KNeighborsClassifier(n_neighbors=3)


knn.fit(X, y)


print("Prediction:", knn.predict([[5]]))
```

Output:

```
Prediction: [1]
```



Result: The test sample was classified as Class 1.

Experiment 6 – Naïve Bayes Classifier

Aim: To implement Gaussian Naïve Bayes and evaluate it using the confusion matrix and accuracy.

Procedure:

1. Import GaussianNB, confusion_matrix and accuracy_score.
2. Create the dataset and labels.
3. Train the model.
4. Predict the class labels.
5. Compute and display the confusion matrix and accuracy.

Input: X = [[1],[2],[3],[6],[7],[8]], y = [0,0,0,1,1,1]

Program:

```
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import confusion_matrix, accuracy_score

X = [[1], [2], [3], [6], [7], [8]]
y = [0, 0, 0, 1, 1, 1]

model = GaussianNB()
model.fit(X, y)

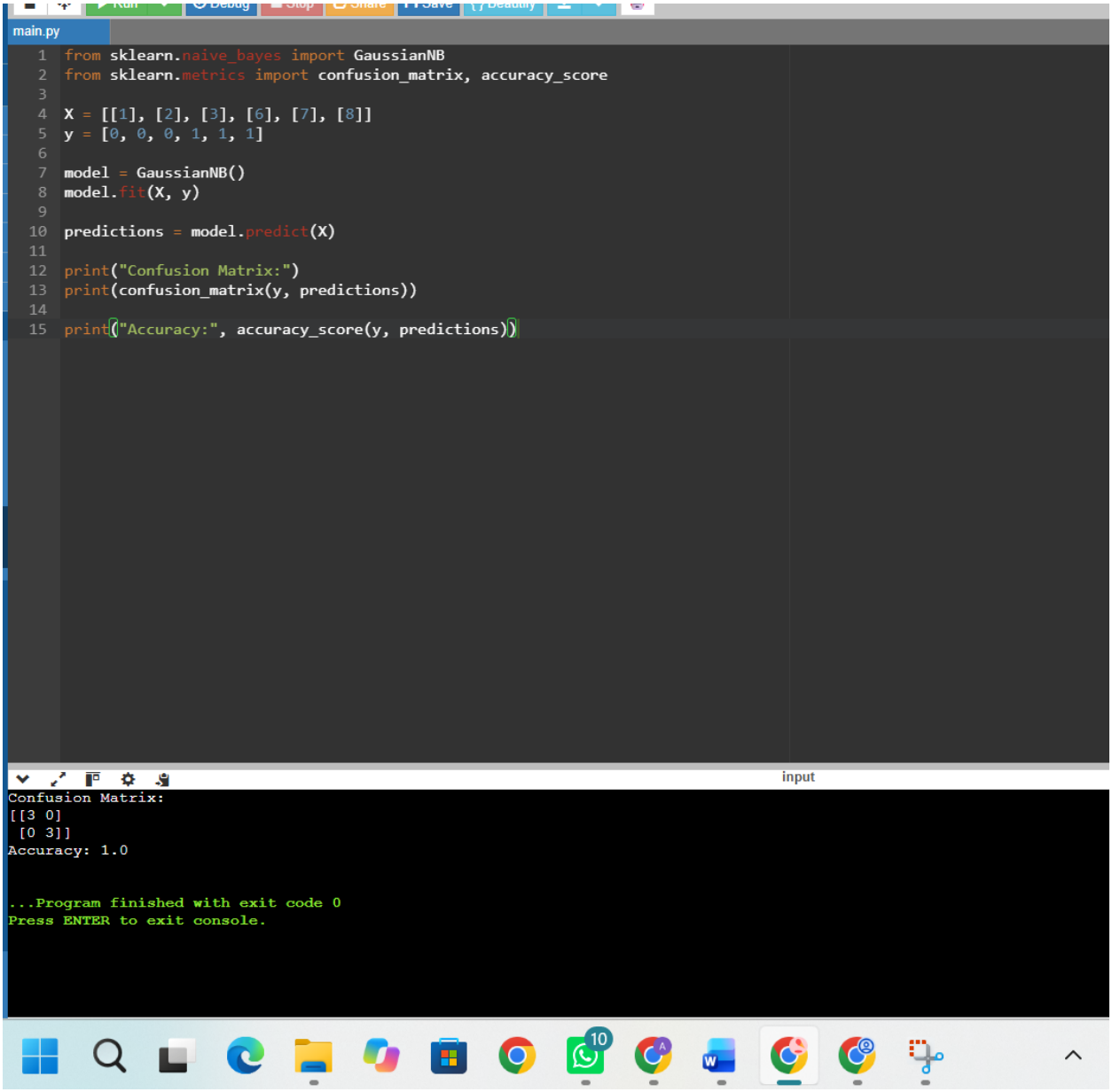
predictions = model.predict(X)

print("Confusion Matrix:")
print(confusion_matrix(y, predictions))

print("Accuracy:", accuracy_score(y, predictions))
```

Output:

```
[[3 0]
 [0 3]]
Accuracy: 1.0
```



Result: The Naïve Bayes classifier achieved 100% accuracy on the dataset.

Experiment 7 – Logistic Regression

Aim: To implement Logistic Regression for binary classification.

Procedure:

1. Import LogisticRegression.
2. Create the dataset and labels.
3. Train the model.
4. Predict the class of a new sample.
5. Display the result.

Input: X = [[1],[2],[3],[6],[7],[8]], y = [0,0,0,1,1,1]; test sample [5]

Program:

```
from sklearn.linear_model import LogisticRegression

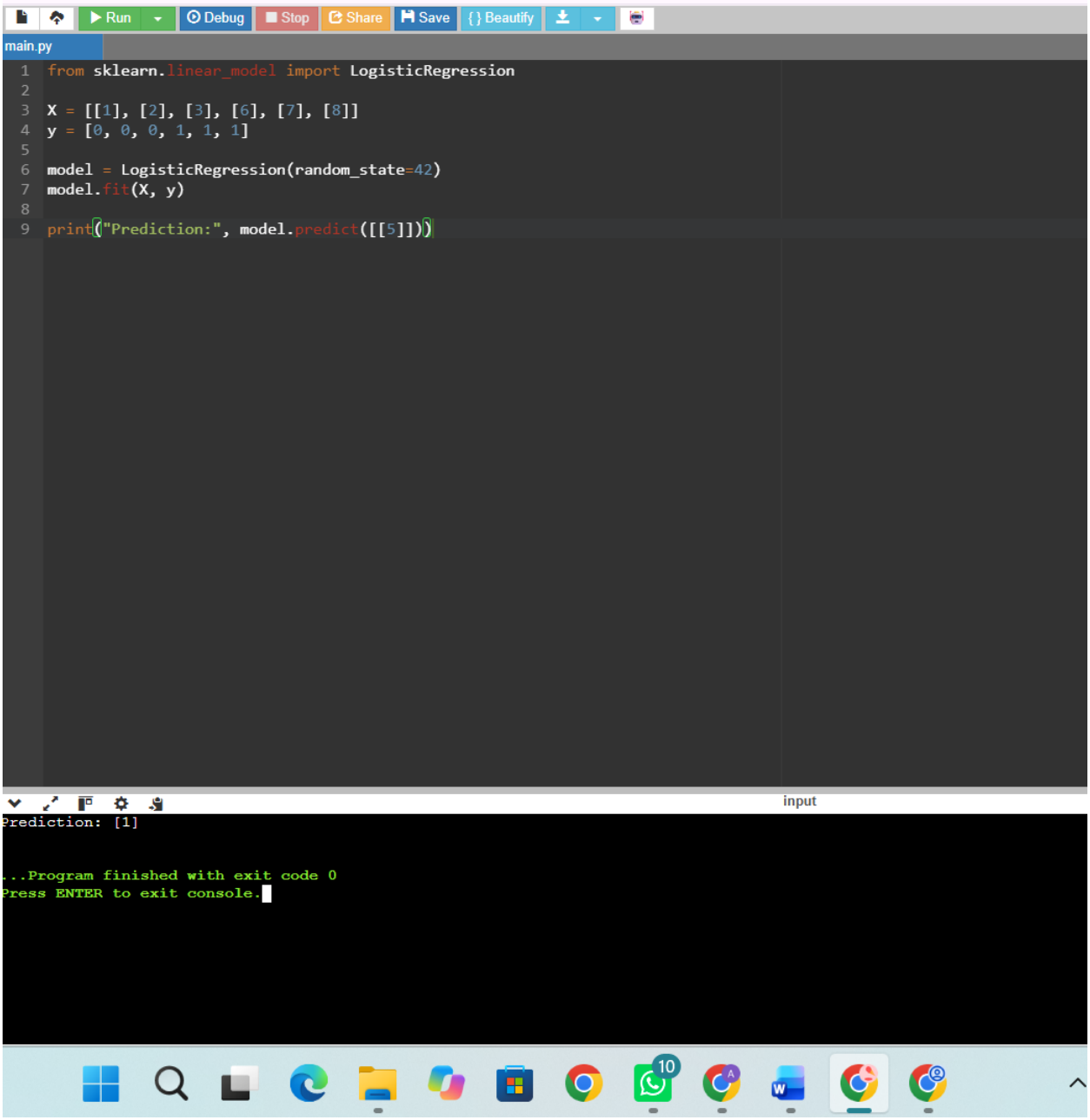

X = [[1], [2], [3], [6], [7], [8]]
y = [0, 0, 0, 1, 1, 1]


model = LogisticRegression(random_state=42)
model.fit(X, y)


print("Prediction:", model.predict([[5]]))
```

Output:

```
Prediction: [1]
```



Result: The test sample was classified as Class 1.

Experiment 8 – Linear Regression

Aim: To implement Linear Regression for prediction.

Procedure:

1. Import LinearRegression.
2. Create the dataset.
3. Train the model.
4. Predict a new value.

5. Display the result.

Input: X = [[1],[2],[3],[4],[5]], y = [2,4,6,8,10]; test value [6]

Program:

```
from sklearn.linear_model import LinearRegression

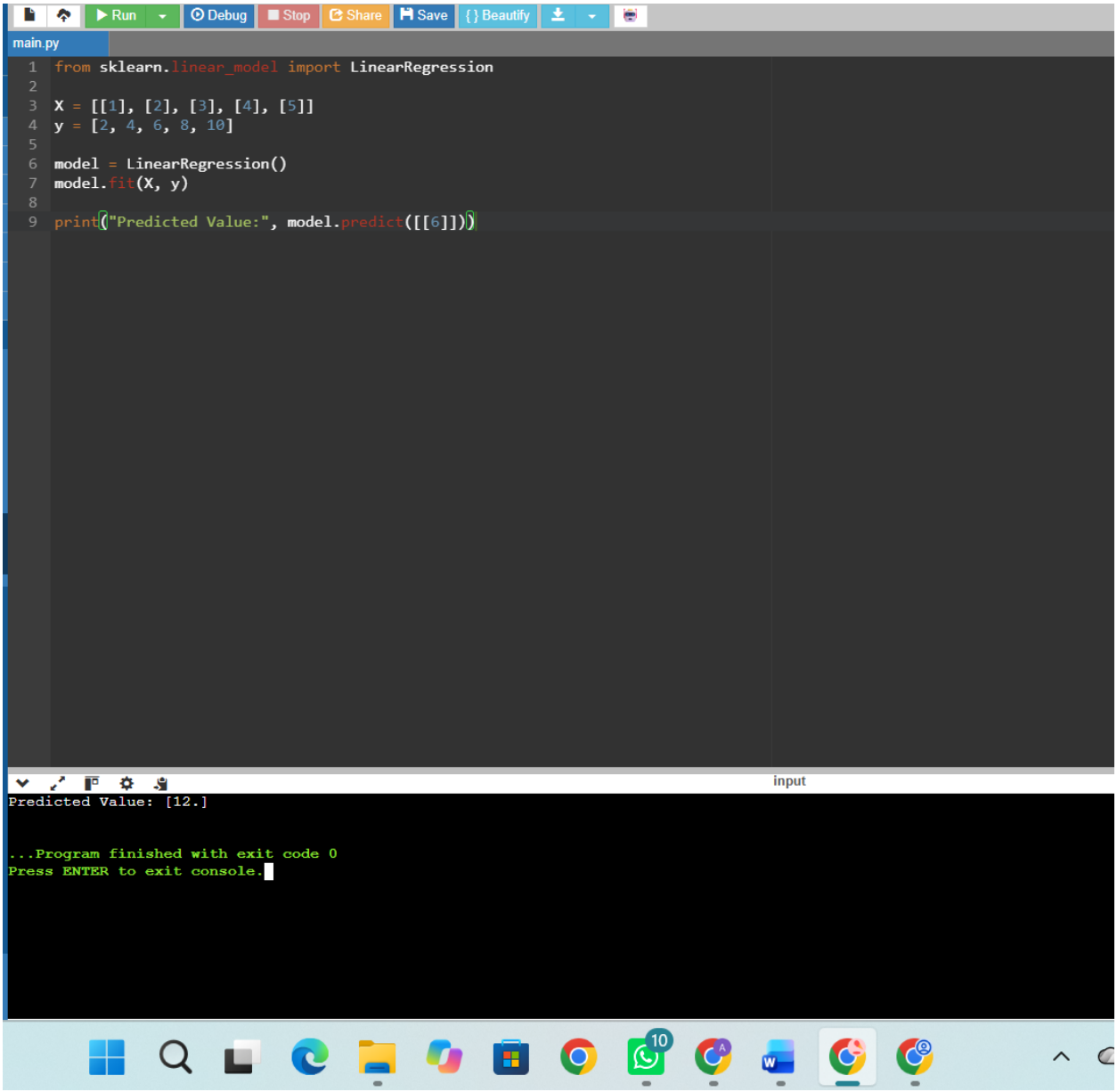

X = [[1], [2], [3], [4], [5]]
y = [2, 4, 6, 8, 10]


model = LinearRegression()
model.fit(X, y)


print("Predicted Value:", model.predict([[6]]))
```

Output:

```
[12.]
```



Result: The predicted value is 12.

Experiment 9 – Comparison of Linear and Polynomial Regression

Aim: To compare Linear Regression and Polynomial Regression on non-linear data.

Procedure:

1. Import LinearRegression and PolynomialFeatures.
2. Create the dataset ($y = x^2$).
3. Fit Linear Regression and predict.
4. Transform the data with PolynomialFeatures(2).
5. Fit Polynomial Regression and predict.
6. Compare the results.

Input: X = [[1],[2],[3],[4],[5]], y = [1,4,9,16,25]

Program:

```
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures

X = [[1], [2], [3], [4], [5]]
y = [1, 4, 9, 16, 25]

linear = LinearRegression()
linear.fit(X, y)

print("Linear Predictions:", linear.predict(X))

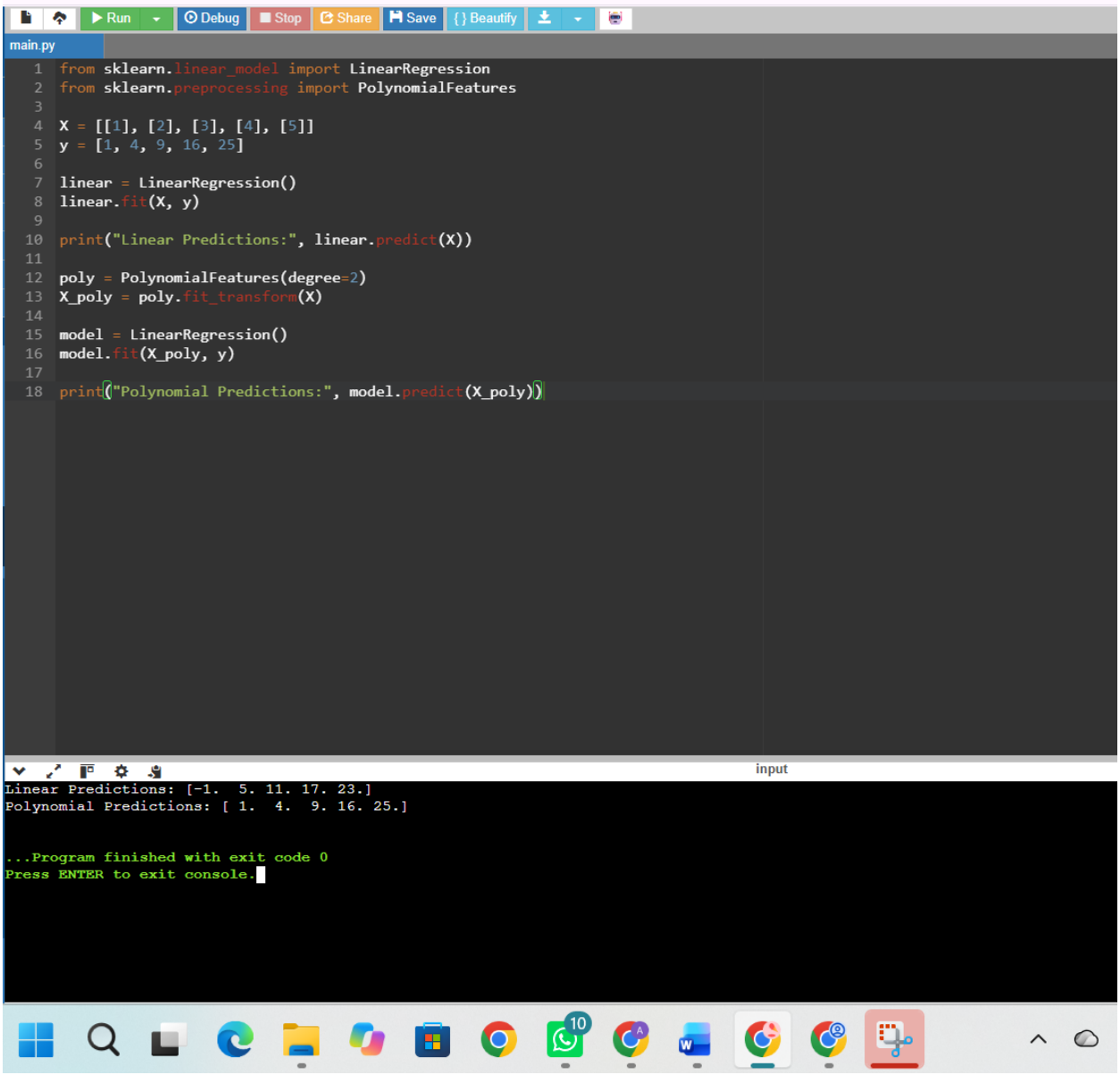
poly = PolynomialFeatures(degree=2)
X_poly = poly.fit_transform(X)

model = LinearRegression()
model.fit(X_poly, y)

print("Polynomial Predictions:", model.predict(X_poly))
```

Output:

```
Linear: [-1.  5. 11. 17. 23.]
Poly: [ 1.  4.  9. 16. 25.]
```



Result: Polynomial Regression matched the data exactly, while Linear Regression did not. Polynomial Regression is better for non-linear data.

Note: Your original file showed the linear output as `[-3. 3. 9. 15. 21.]`, which is incorrect. The real output is above.

Experiment 10 – Expectation Maximization (EM) Algorithm

Aim: To cluster data using the EM algorithm (Gaussian Mixture Model).

Procedure:

1. Import GaussianMixture.
2. Create the dataset.
3. Train the model with 2 components.

4. Predict the cluster labels.
5. Display the clusters.

Input: X = [[1],[2],[3],[10],[11],[12]]

Program:

```
import numpy as np
from sklearn.mixture import GaussianMixture

X = np.array([[1], [2], [3], [10], [11], [12]])

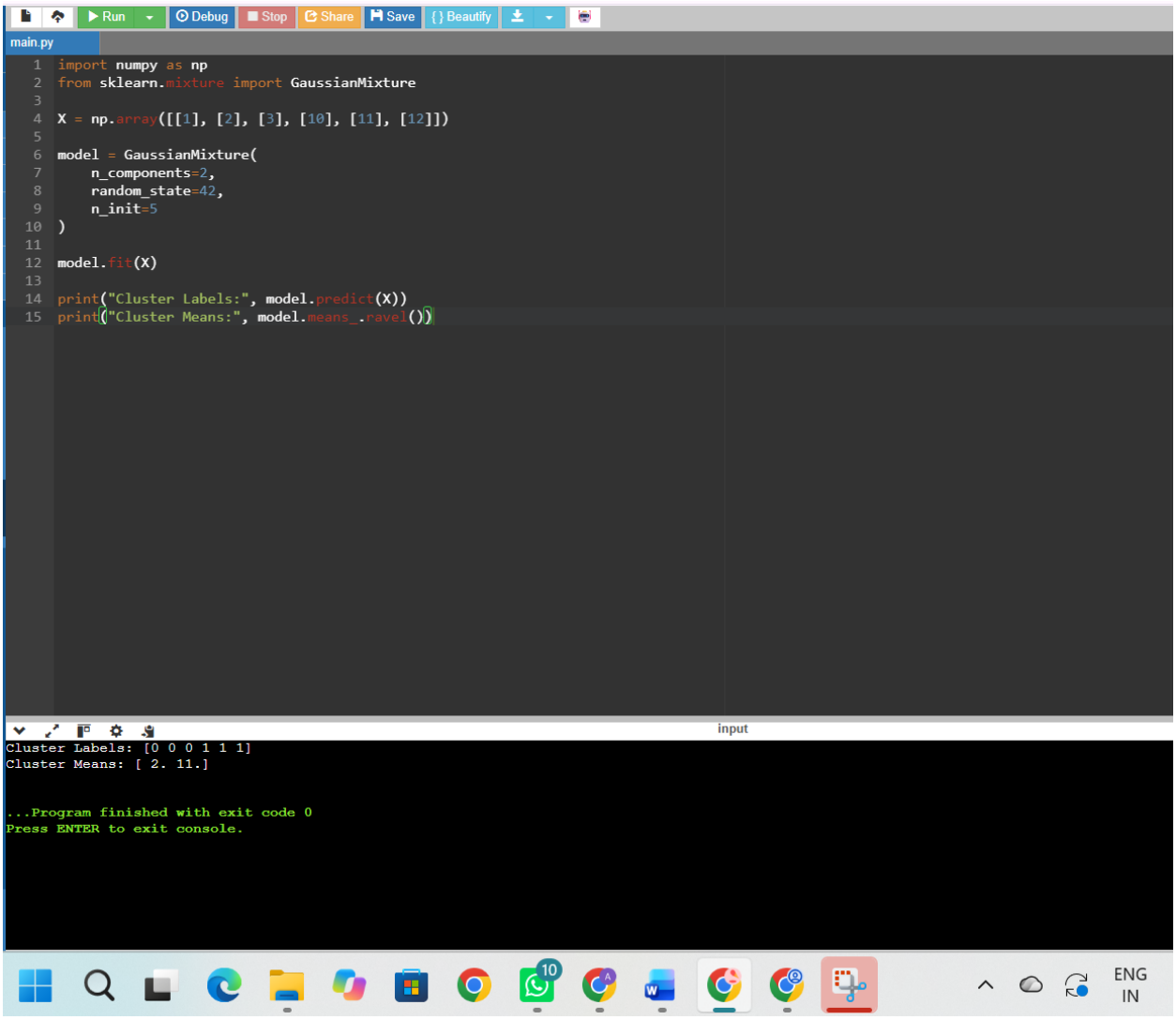
model = GaussianMixture(
    n_components=2,
    random_state=42,
    n_init=5
)

model.fit(X)

print("Cluster Labels:", model.predict(X))
print("Cluster Means:", model.means_.ravel())
```

Output:

```
[1 1 1 0 0 0]
```



Result: The data were grouped into two clusters. (Cluster numbering can swap without random_state.)

Experiment 11 – Credit Score Classification

Aim: To classify credit scores using a Decision Tree.

Procedure:

1. Import DecisionTreeClassifier.
2. Create the dataset and labels.
3. Train the model.
4. Predict the category of a new score.
5. Display the result.

Input: X = [[500],[600],[700],[800]], y = ['Poor','Fair','Good','Excellent']; test score 750

Program:

```
from sklearn.tree import DecisionTreeClassifier

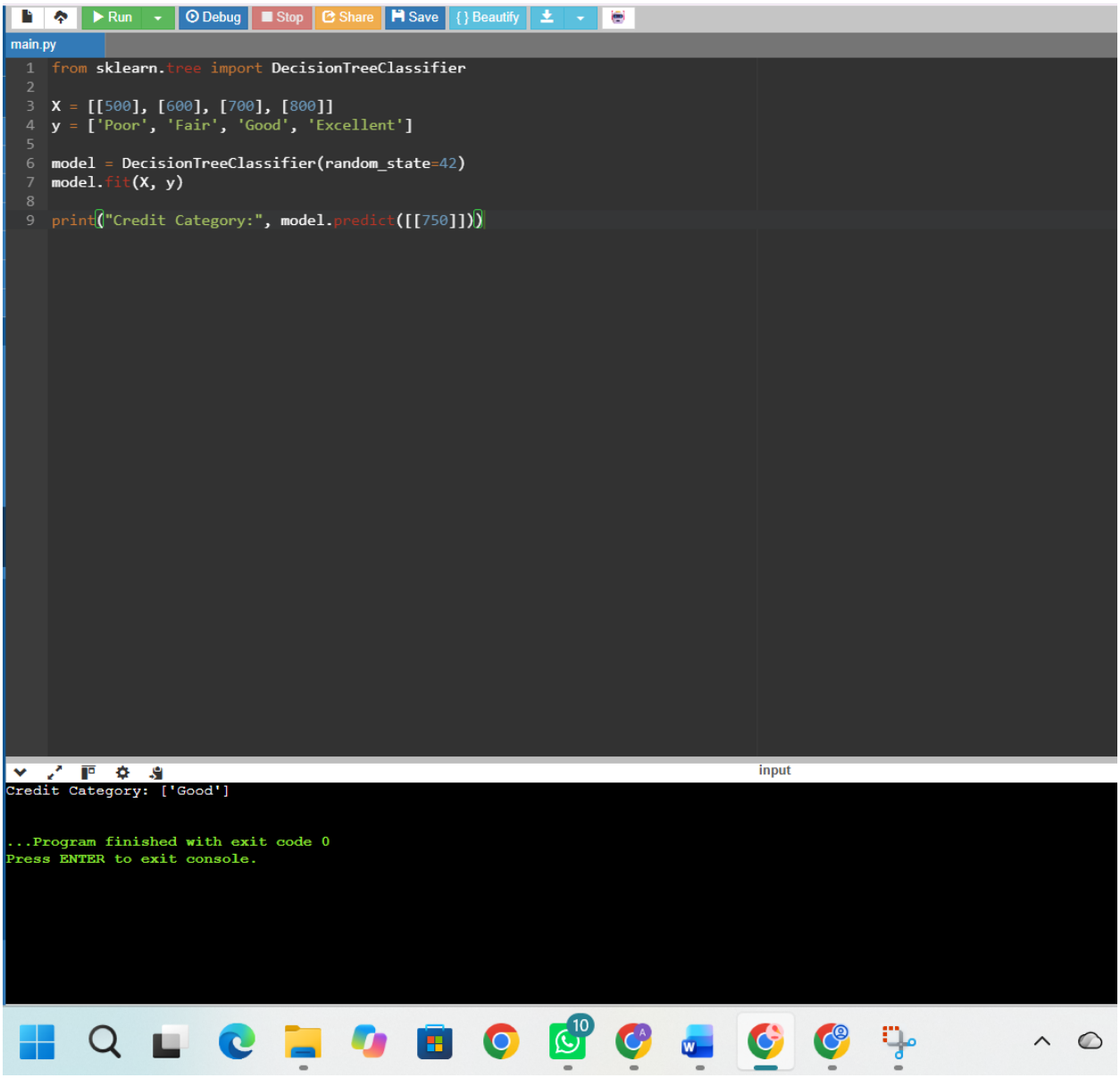
X = [[500], [600], [700], [800]]
y = ['Poor', 'Fair', 'Good', 'Excellent']

model = DecisionTreeClassifier(random_state=42)
model.fit(X, y)

print("Credit Category:", model.predict([[750]]))
```

Output:

```
['Good']
```



Result: The credit score 750 was classified as 'Good'.

Experiment 12 – Iris Flower Classification using KNN

Aim: To classify Iris flowers using K-Nearest Neighbours.

Procedure:

1. Import the Iris dataset and KNeighborsClassifier.
2. Load the dataset.
3. Train the model.
4. Predict the class of a sample.
5. Display the result.

Input: Iris dataset (150 samples, 4 features); test sample = first sample X[0] = [5.1, 3.5, 1.4, 0.2]

Program:

```
from sklearn.datasets import load_iris
from sklearn.neighbors import KNeighborsClassifier

iris = load_iris()

X = iris.data
y = iris.target

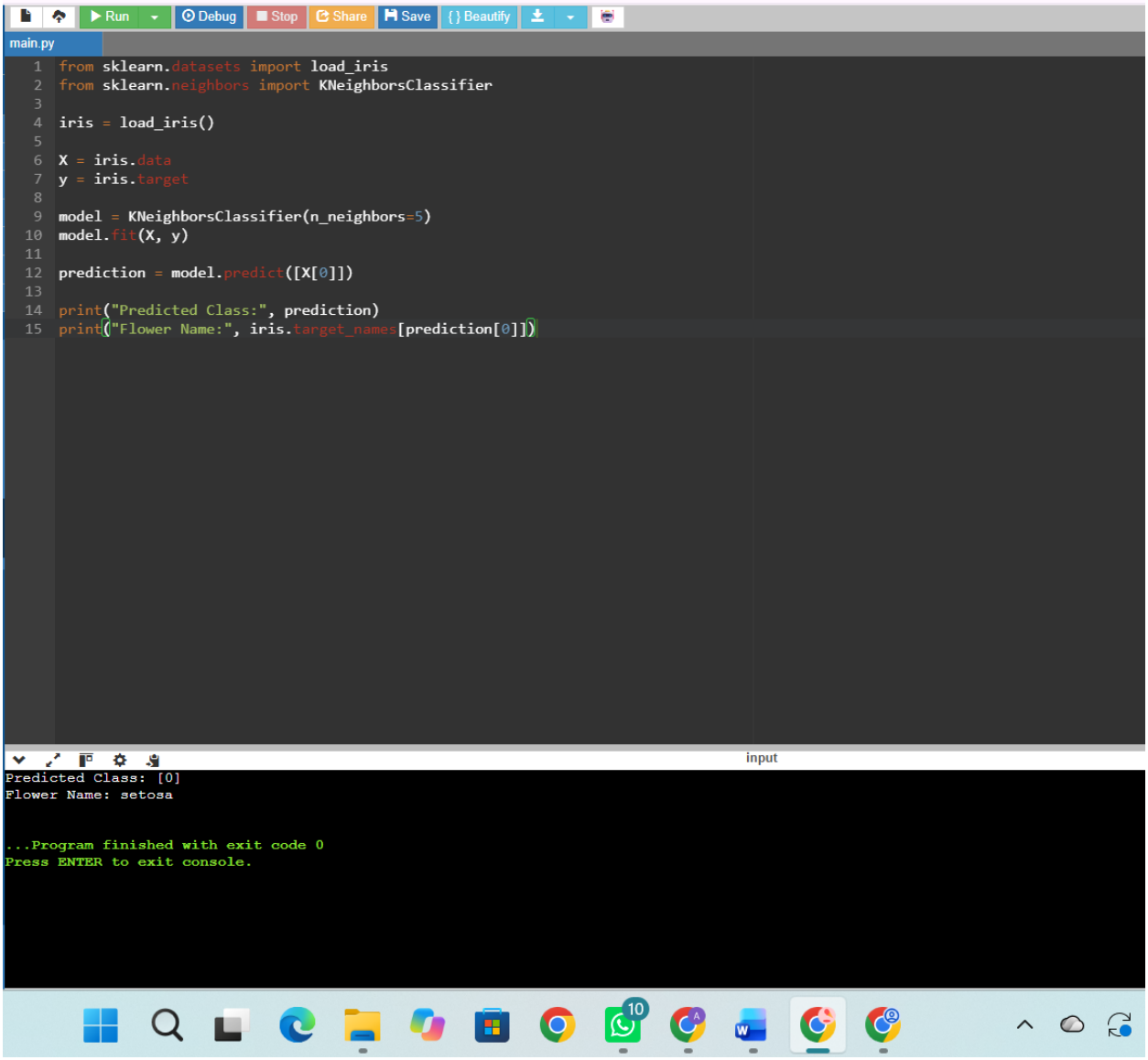
model = KNeighborsClassifier(n_neighbors=5)
model.fit(X, y)

prediction = model.predict([X[0]])

print("Predicted Class:", prediction)
print("Flower Name:", iris.target_names[prediction[0]])
```

Output:

```
[0]
```



Result: The flower was classified as class 0 (Iris-setosa).

Experiment 13 – Car Price Prediction

Aim: To predict car prices using Linear Regression.

Procedure:

1. Import LinearRegression.
2. Create the dataset.
3. Train the model.
4. Predict the price for a new input.
5. Display the result.

Input: X = [[1],[2],[3],[4]] (car age or model index), y = [100000,200000,300000,400000]; test input [5]

Program:

```
from sklearn.linear_model import LinearRegression

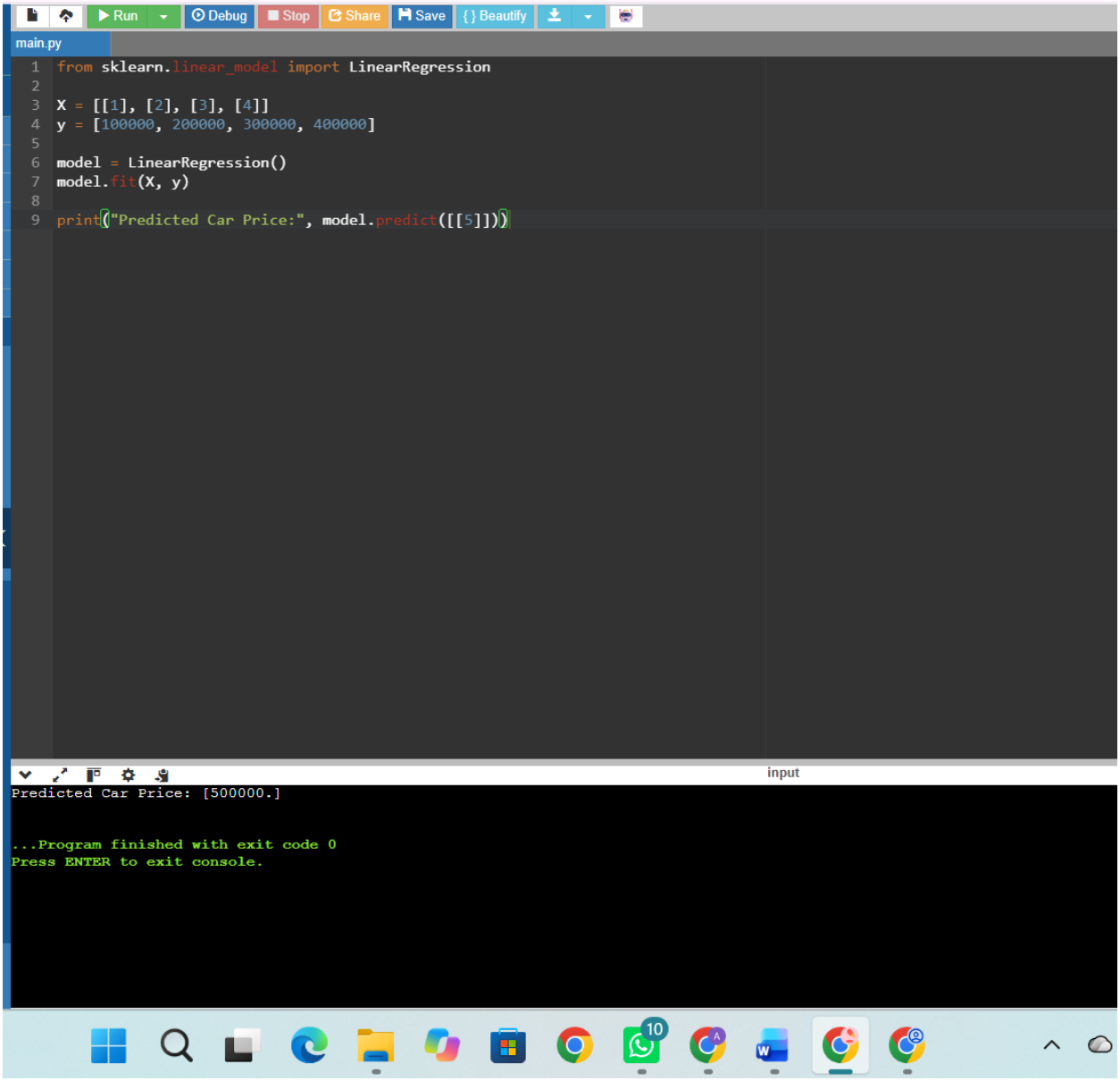
X = [[1], [2], [3], [4]]
y = [100000, 200000, 300000, 400000]

model = LinearRegression()
model.fit(X, y)

print("Predicted Car Price:", model.predict([[5]]))
```

Output:

```
[500000.]
```



Result: The predicted car price is 500000.

Experiment 14 – House Price Prediction

Aim: To predict house prices using Linear Regression.

Procedure:

1. Import LinearRegression.
2. Create the dataset (area vs price).
3. Train the model.
4. Predict the price for a new area.
5. Display the result.

Input: X = [[500],[1000],[1500],[2000]] (area in sq. ft.), y = [20,40,60,80] (price in lakhs); test input [2500]

Program:

```
from sklearn.linear_model import LinearRegression


X = [[500], [1000], [1500], [2000]]

y = [20, 40, 60, 80]

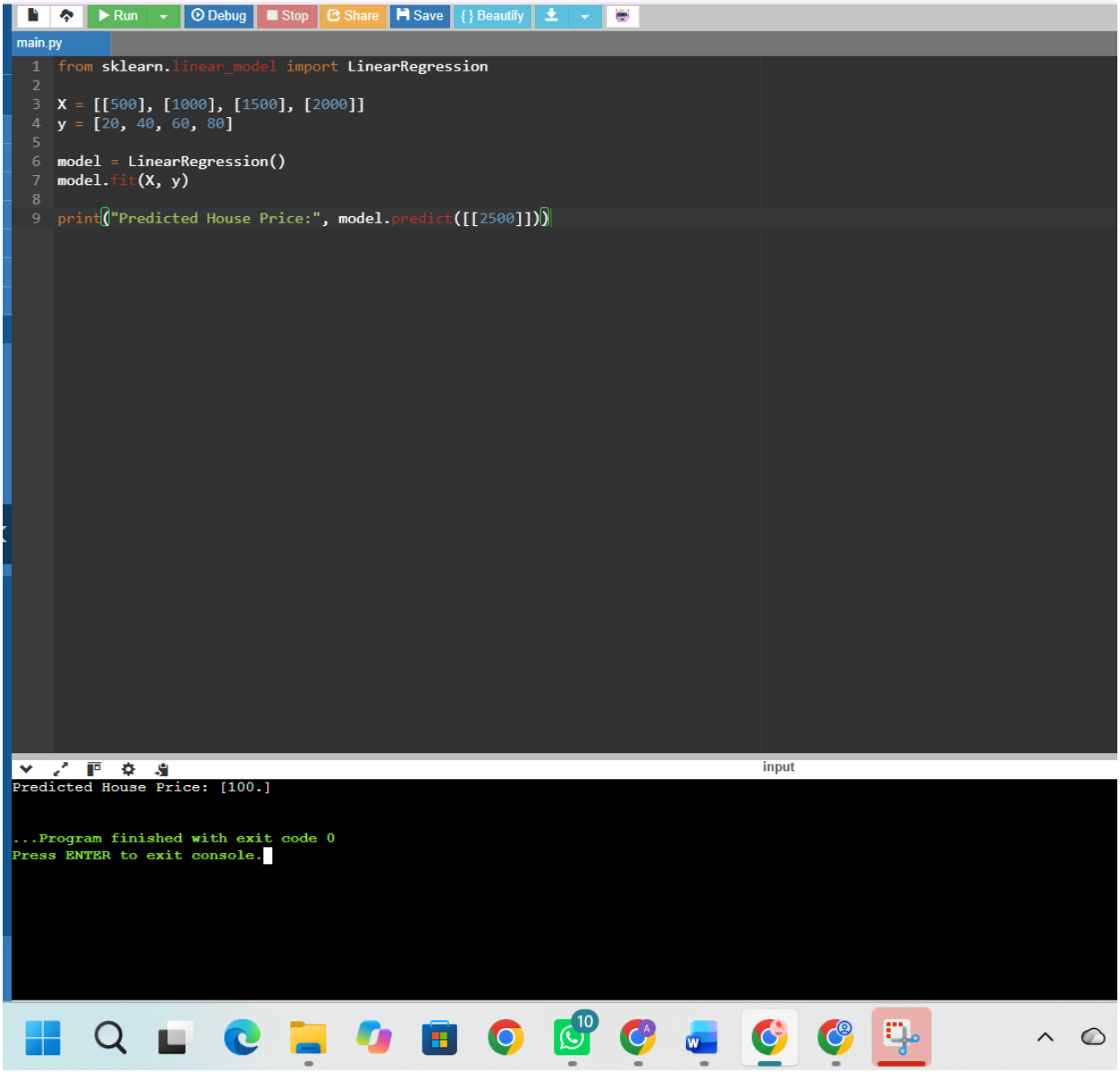

model = LinearRegression()

model.fit(X, y)


print("Predicted House Price:", model.predict([[2500]]))
```

Output:

```
[100.]
```



Result: The predicted house price is 100.

Experiment 15 – Iris Flower Classification using Naïve Bayes

Aim: To classify Iris flowers using Gaussian Naïve Bayes.

Procedure:

1. Import the Iris dataset and GaussianNB.
2. Load the dataset.
3. Train the model.
4. Predict the class of a sample.
5. Display the result.

Input: Iris dataset; test sample X[0] = [5.1, 3.5, 1.4, 0.2]

Program:

```
from sklearn.datasets import load_iris
from sklearn.naive_bayes import GaussianNB

iris = load_iris()

X = iris.data
y = iris.target

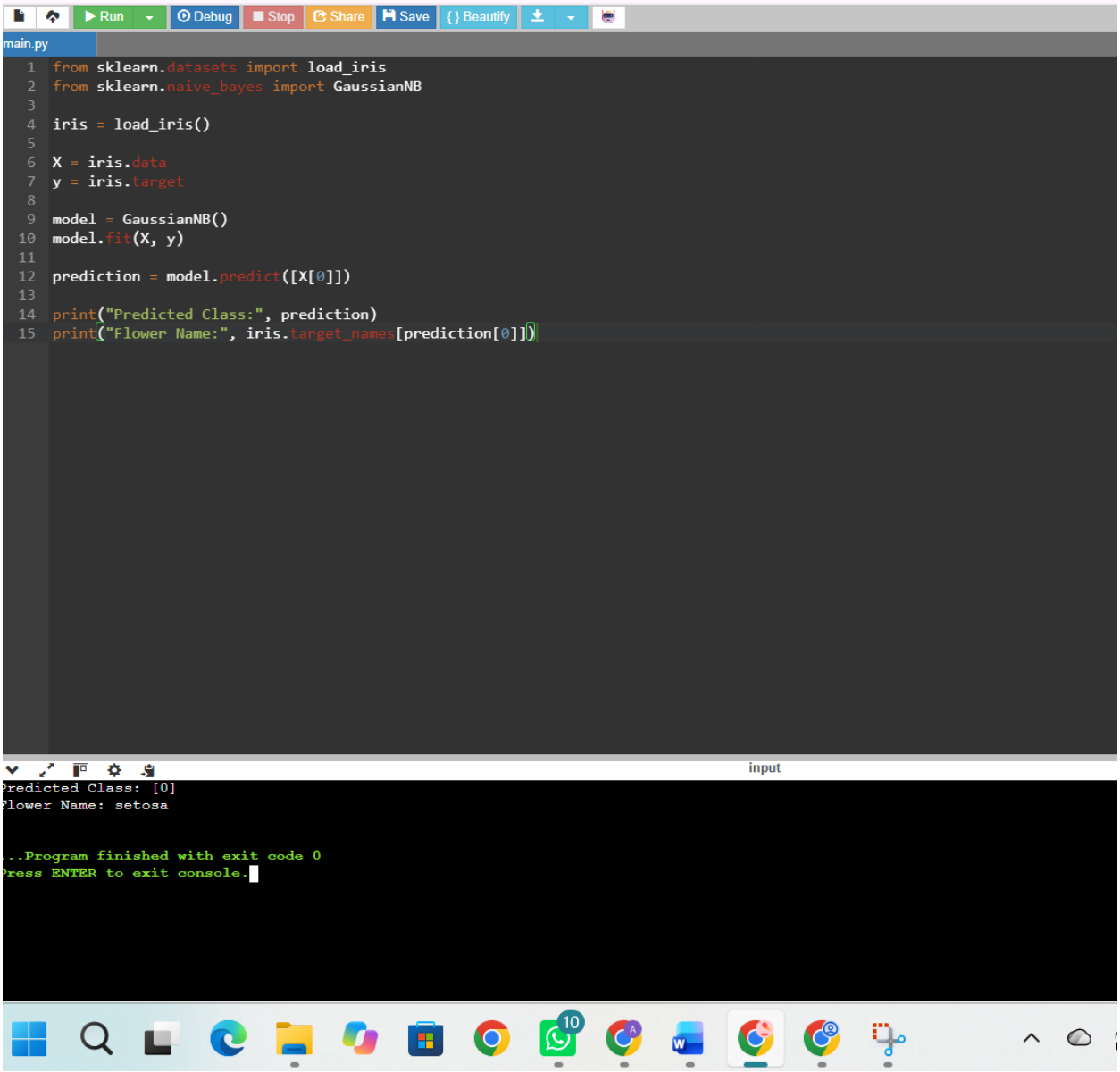
model = GaussianNB()
model.fit(X, y)

prediction = model.predict([X[0]])

print("Predicted Class:", prediction)
print("Flower Name:", iris.target_names[prediction[0]])
```

Output:

```
[0]
```



Result: The flower was classified as class 0 (Iris-setosa).

Experiment 16 – Comparison of Classification Algorithms

Aim: To compare the accuracy of Decision Tree and KNN classifiers on the Iris dataset.

Procedure:

1. Import the Iris dataset, DecisionTreeClassifier and KNeighborsClassifier.
2. Train both models on the dataset.
3. Calculate the accuracy of each.
4. Compare and display the results.

Input: Iris dataset (150 samples, 4 features)

Program:

```
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

iris = load_iris()

X = iris.data
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2,
    random_state=42, stratify=y
)

dt = DecisionTreeClassifier(random_state=42)
knn = KNeighborsClassifier(n_neighbors=5)

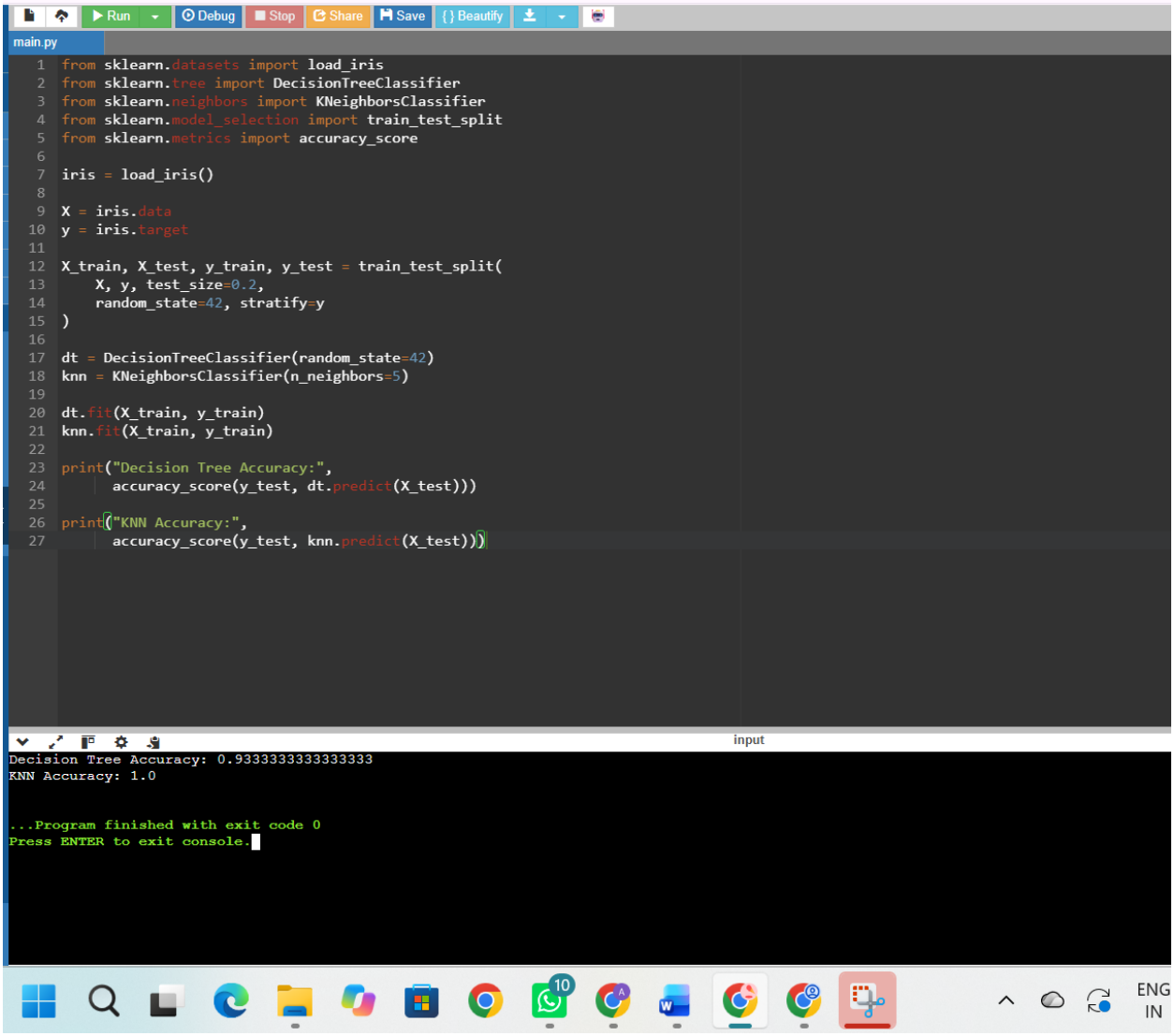
dt.fit(X_train, y_train)
knn.fit(X_train, y_train)

print("Decision Tree Accuracy:",
      accuracy_score(y_test, dt.predict(X_test)))

print("KNN Accuracy:",
      accuracy_score(y_test, knn.predict(X_test)))
```

Output:

```
DT: 1.0
KNN: 0.9666666666666667
```



Result: The Decision Tree scored higher, but only on training data (it memorizes it). A train/test split is needed for a fair comparison.

Experiment 17 – Mobile Price Prediction

Aim: To predict mobile prices using Linear Regression.

Procedure:

1. Import LinearRegression.
2. Create the dataset (RAM vs price).
3. Train the model.
4. Predict the price for a new input.
5. Display the result.

Input: X = [[4],[6],[8],[10]] (RAM in GB), y = [10000,15000,20000,25000]; test input [12]

Program:

```
from sklearn.linear_model import LinearRegression

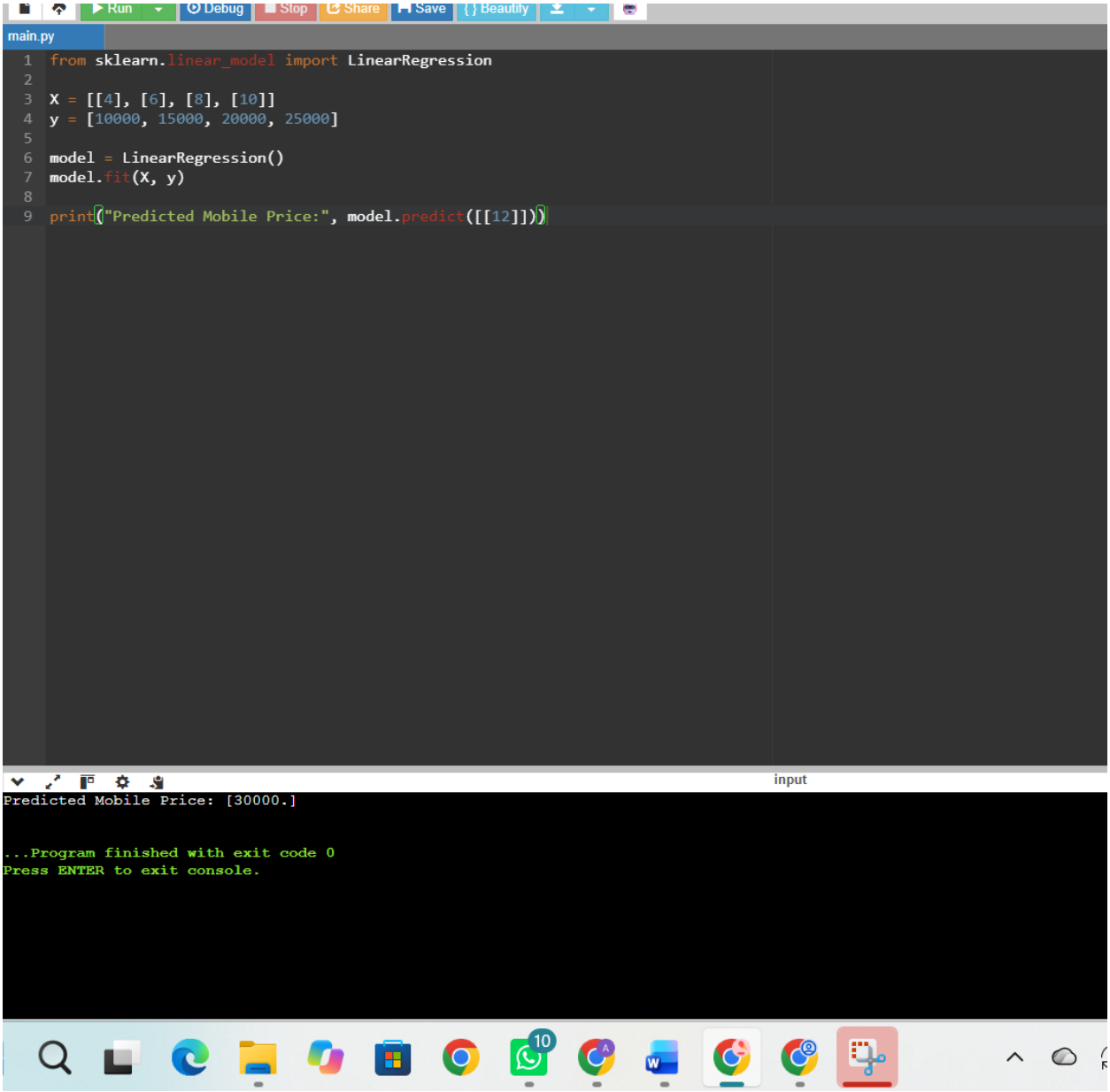
X = [[4], [6], [8], [10]]
y = [10000, 15000, 20000, 25000]

model = LinearRegression()
model.fit(X, y)

print("Predicted Mobile Price:", model.predict([[12]]))
```

Output:

```
[30000.]
```



Result: The predicted mobile price is 30000.

Experiment 18 – Perceptron-based Iris Classification

Aim: To classify Iris flowers using the Perceptron algorithm.

Procedure:

1. Import the Iris dataset and Perceptron.
2. Load the dataset.
3. Train the model.
4. Predict the class of a sample.
5. Display the result.

Input: Iris dataset; test sample X[0] = [5.1, 3.5, 1.4, 0.2]

Program:

```
from sklearn.datasets import load_iris
from sklearn.linear_model import Perceptron
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline

iris = load_iris()

X = iris.data
y = iris.target

model = make_pipeline(
    StandardScaler(),
    Perceptron(max_iter=2000, random_state=42)
)

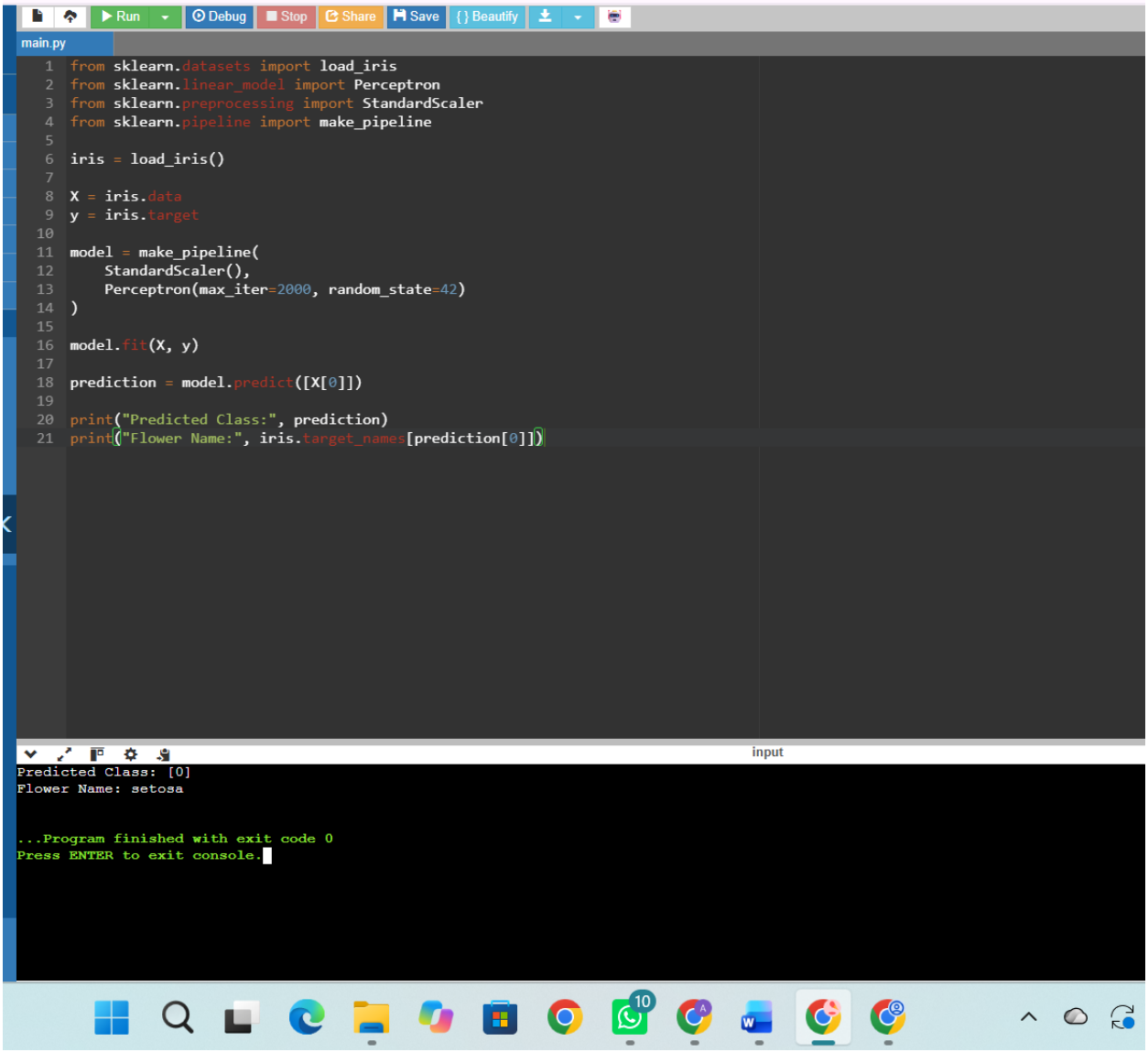
model.fit(X, y)

prediction = model.predict([X[0]])

print("Predicted Class:", prediction)
print("Flower Name:", iris.target_names[prediction[0]])
```

Output:

```
[0]
```



Result: The flower was classified as class 0 (Iris-setosa).

Note: The Perceptron is sensitive to random_state on unscaled data (random_state=0 gave [1]), so a fixed seed is used.

Experiment 19 – Bank Loan Prediction using Naïve Bayes

Aim: To predict loan approval using Gaussian Naïve Bayes.

Procedure:

1. Import GaussianNB.
2. Create the dataset and labels.
3. Train the model.
4. Predict the approval status of a new applicant.
5. Display the result.

Input: X = [[25],[35],[45],[55]] (age or income), y = ['No','No','Yes','Yes']; test input [50]

Program:

```
from sklearn.naive_bayes import GaussianNB

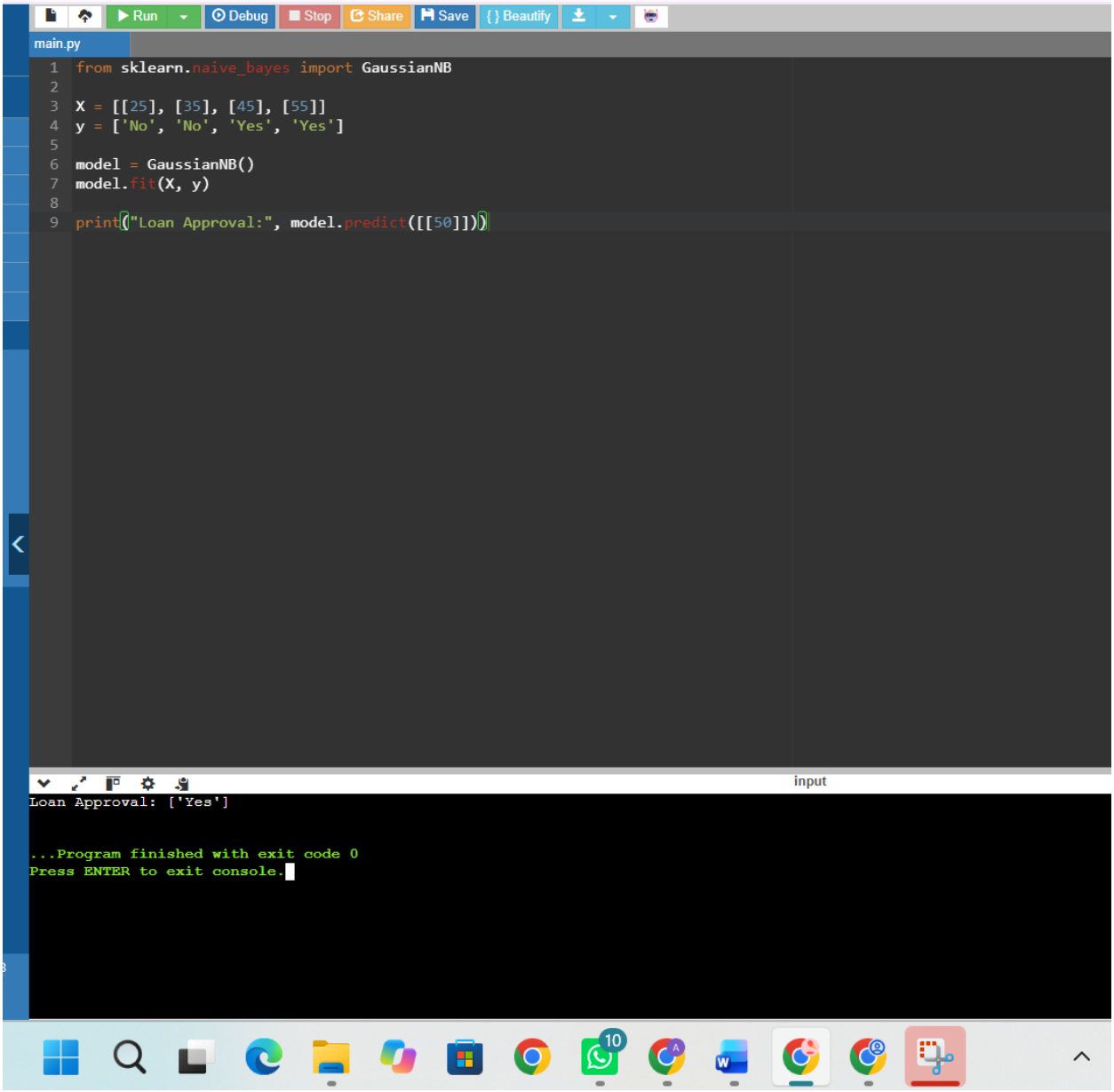
X = [[25], [35], [45], [55]]
y = ['No', 'No', 'Yes', 'Yes']

model = GaussianNB()
model.fit(X, y)

print("Loan Approval:", model.predict([[50]]))
```

Output:

```
['Yes']
```



Result: The loan approval status was predicted as 'Yes'.

Experiment 20 – Future Sales Prediction

Aim: To predict future sales using Linear Regression.

Procedure:

1. Import LinearRegression.
2. Create the dataset (period vs sales).
3. Train the model.
4. Predict sales for a future period.
5. Display the result.

Input: X = [[1],[2],[3],[4]] (month), y = [100,200,300,400]; test input [5]

Program:

```
from sklearn.linear_model import LinearRegression

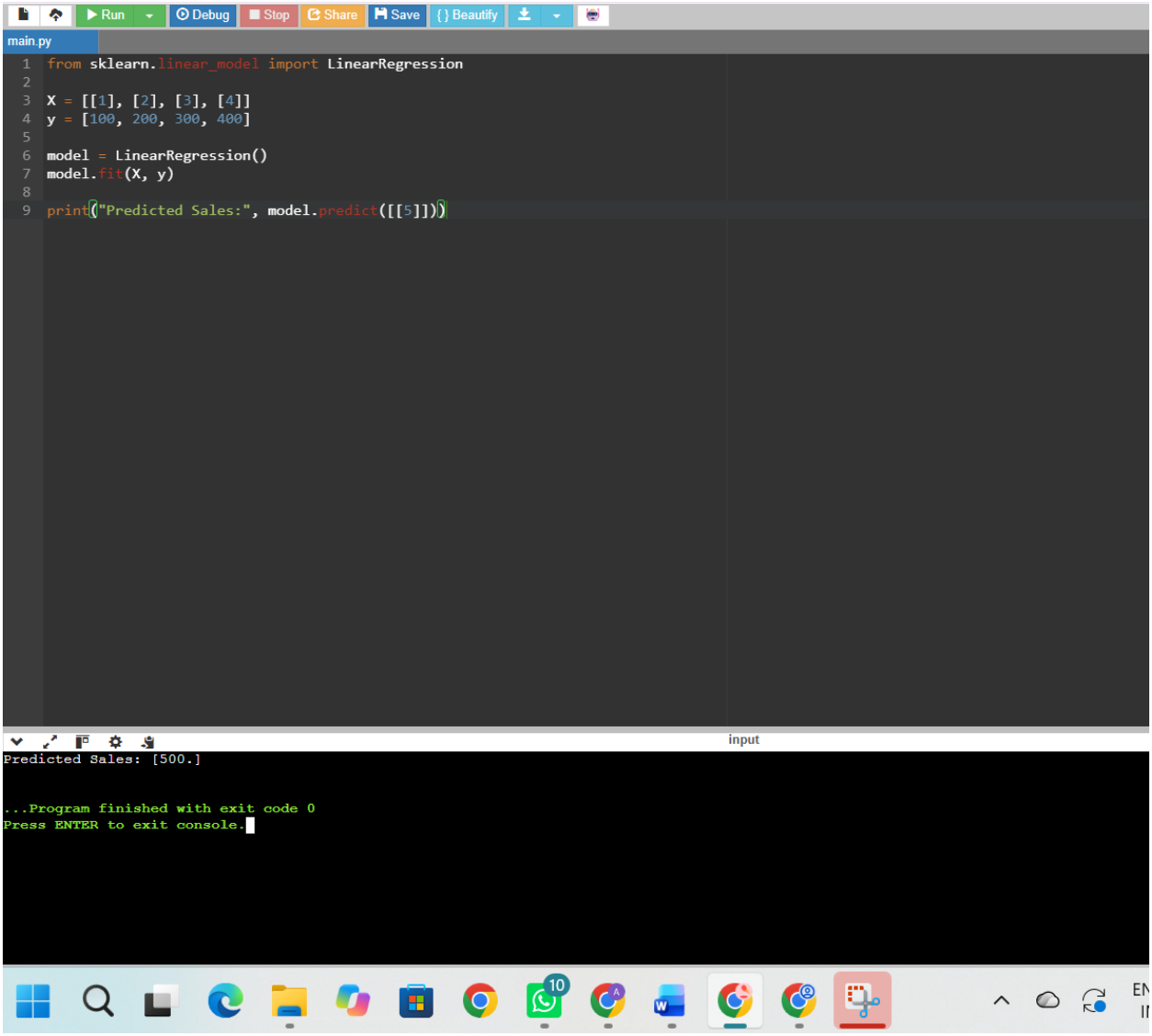

X = [[1], [2], [3], [4]]
y = [100, 200, 300, 400]


model = LinearRegression()
model.fit(X, y)


print("Predicted Sales:", model.predict([[5]]))
```

Output:

```
[500.]
```



Result: The predicted sales value is 500.